

Portfolio Rebalancer Developer Guide

Architecture, services, deployment, and extension guide

<https://github.com/johnlatona-hoh/portfolio-rebalancer>

1. Project Overview

The Portfolio Rebalancer is a personal, tax-aware investment rebalancing tool built as a full-stack web application. It ingests brokerage CSV exports, runs a pure-Python rebalancing engine, projects portfolio growth via Monte Carlo simulation, and optionally surfaces Gemini AI tax-location advice.

Component	Technology
Backend API	FastAPI (Python 3.13), async SQLAlchemy, asyncpg
Database	Supabase PostgreSQL (shared with MusicCollection app)
Frontend	React 18, Vite, TypeScript, TailwindCSS, recharts
AI	Google Gemini 2.5 Flash via google-genai SDK
Prices	Yahoo Finance v8 chart endpoint (no API key required)
Backend host	Render.com (free tier, sleeps after 15 min idle)
Frontend host	Vercel (auto-deploys on git push to main)
CI	GitHub Actions (.github/workflows/ci.yml)
Repo	github.com/johnlatona-hoh/portfolio-rebalancer, branch: main

Privacy Architecture

Portfolio working data lives entirely in the browser (React context, no server calls).

Server only stores: (1) ticker tag map (shared, public), (2) encrypted snapshot payloads (PIN-keyed Fernet, PII-light).

No account names, dollar amounts, or PII leave the browser unless the user saves a snapshot.

2. Live Service URLs

Service	URL
Frontend (Vercel)	https://frontend-sooty-three-ybjekuhhod.vercel.app
Backend API (Render)	https://portfolio-rebalancer-tc4h.onrender.com
API docs (Swagger)	https://portfolio-rebalancer-tc4h.onrender.com/docs
GitHub repo	https://github.com/johnlatona-hoh/portfolio-rebalancer

3. Local Development

Python 3.13 is **required**. Python 3.14 has no prebuilt asyncpg or pydantic-core wheels and source builds fail. Backend source files must stay **ASCII-only**: Windows imports .py files as cp1252, so non-ASCII punctuation in string literals becomes mojibake in API responses.

3.1 Backend Setup

```
cd backend
py -3.13 -m venv venv
.\venv\Scripts\python.exe -m pip install -r requirements.txt
copy .env.example .env # then fill in DATABASE_URL etc.
.\venv\Scripts\python.exe -m uvicorn main:app --reload
# API at http://localhost:8000 | Swagger at /docs
```

3.2 Frontend Setup

```
cd frontend
npm install
npm run dev # http://localhost:5173 (proxies /api/* -> :8000)
npx tsc --noEmit # type-check only (pre-commit gate)
npm run build # production build
```

3.3 Seed Ticker Data

Run after first deploy or whenever seed_data.py is updated:

```
cd backend && python seed_tickers.py
```

3.4 Run Tests

Backend has full unit tests for the rebalancing and projection engines:

```
.\venv\Scripts\python.exe -m pytest tests/ -q
```

4. Environment Variables

4.1 Backend (.env / Render dashboard)

Variable	Purpose
DATABASE_URL	Async PostgreSQL URL: postgresql+asyncpg://user:pass@host:5432/dbname

GEMINI_API_KEY	Optional. Enables AI advisor and ticker-tag suggestions. No-ops when absent.
SNAPSHOT_ENCRYPTION_KEY	Fernet key for encrypting snapshots. Generate with: <code>python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"</code>
CORS_ORIGINS	Comma-separated allowed origins, e.g. <code>https://yourdomain.vercel.app</code>

4.2 Frontend (Vercel environment settings)

Variable	Purpose
VITE_API_URL	Full URL of the Render backend, e.g. <code>https://portfolio-rebalancer-tc4h.onrender.com</code>

5. Architecture Deep-Dive

5.1 Backend Services

All backend code lives in `backend/`. Key files:

File	Responsibility
<code>main.py</code>	FastAPI app, CORS, lifespan (<code>create_tables</code> + <code>run_migrations</code>), router includes.
<code>constants.py</code>	Canonical vocabulary: 9 <code>ASSET_CLASSES</code> , <code>SUBCLASS</code> (parent/tax), <code>RETURN_ASSUMPTIONS</code> (mean/stddev for Monte Carlo), <code>MAX_DRAWDOWN_PCT</code> , <code>CORRELATION</code> matrix, <code>FEE_ASSUMPTIONS</code> .
<code>models.py</code>	SQLAlchemy ORM: <code>TickerTag</code> , <code>Snapshot</code> , <code>User</code> , <code>PriceCache</code> , <code>AI</code> Cache. All tables prefixed <code>rebalancer_</code> .
<code>schemas.py</code>	Pydantic v2 request/response models. Keep in sync with frontend <code>api/client.ts</code> .
<code>database.py</code>	Async engine + session factory (<code>AsyncSessionLocal</code>). Reads <code>DATABASE_URL</code> .
<code>seed_data.py</code>	~66 common ETF/index fund tags with expense ratios. Idempotent upsert.
<code>seed_tickers.py</code>	Entry point: calls <code>seed_data</code> functions. Run manually after deploy.

5.2 Backend Routers

Router	Endpoints & Purpose
portfolio.py	POST /portfolio/analyze - full rebalance analysis (allocation, deltas, trades, grade, tax-loss harvest). POST /portfolio/project - Monte Carlo + deterministic projection.
tags.py	GET /tags, POST /tags (upsert), POST /tags/auto (classify by broker description), POST /tags/suggest (Gemini fallback). load_tags() builds the ticker->tag dict for the engine.
prices.py	GET /prices?tickers=VTI,BND - fetches live prices via Yahoo Finance v8 (24h cache).
snapshots.py	POST /snapshots (save encrypted), POST /snapshots/load (PIN-authenticated load), GET /snapshots (list by user).
users.py	POST /users/register, POST /users/login (email + salted PIN hash).
advisor.py	POST /advisor/insights - Gemini AI tax-location analysis, AICache-backed.

5.3 Backend Pure-Python Services

Service	What it does
services/rebalance.py	Core rebalancing engine (no DB/network). roll_up(), compute_deltas(), target_composition() + plan_trades() (per-account, cash-neutral, tax-aware via PLACEMENT map), location_grade(), _tax_loss_harvest(), within_band_classes(), analyze() orchestrator.
services/projections.py	Monte Carlo (1000 paths) + deterministic FV projection. stdlib only (no numpy). Takes fee_drag and monthly_contribution. Returns percentile fan + points.
services/prices.py	Yahoo Finance v8 chart endpoint fetch. 24h PriceCache. Graceful per-ticker failure. Note: Stooq and Yahoo v7 are blocked from Render's IP.
services/classify.py	Description-based ticker classifier (no AI). Maps broker Description + Asset Type text to a sub-class + tax-efficiency. Used for auto-classify on CSV upload.

services/ai.py	Gemini wrapper (google-genai SDK, gemini-2.5-flash). _generate_with_retry() with 429 backoff. portfolio_insights(), suggest_ticker_tag(). No-ops without GEMINI_API_KEY.
services/crypto.py	Fernet encrypt/decrypt for snapshot payloads + salted HMAC-SHA256 PIN hash.
services/csv_template.py	CSV column contract + validator for the simple template format.

5.4 Rebalancing Engine Details

The engine in services/rebalance.py is pure Python (no DB or network calls) so it is trivially unit-testable. Key functions:

- **roll_up(holdings, tags)** - aggregates holding values into blended_by_class dict.
- **compute_deltas(blended, total, targets)** - computes dollar delta vs target for each class.
- **within_band_classes(blended, total, targets, band_pct)** - returns set of classes within +/- band of target; these are 'frozen' (no trade generated).
- **target_composition(holdings, targets, tags, total, frozen)** - per-account, cash-neutral fill using PLACEMENT priority order. Frozen classes are pinned to current holdings.
- **plan_trades(holdings, targets, tags, total, gain_aversion, frozen)** - generates BUY/SELL trade list with estimated gains, respecting gain_aversion weighting.
- **location_grade(by_account, tags)** - 1-10 asset location score.
- **_tax_loss_harvest(holdings, tags)** - scans taxable lots with cost_basis > 0 and current_value < cost_basis.
- **analyze(holdings, targets, tags, gain_aversion, drift_band_pct)** - orchestrates all of the above and returns the complete AnalyzeResponse dict.

Asset placement order (PLACEMENT map): tax-deferred accounts fill tax-inefficient income first (bonds, REITs, gold, crypto); tax-free (Roth) fills highest-growth first (crypto, equity); taxable fills tax-efficient first (munis, equity).

5.5 Frontend Structure

Path	Purpose
api/client.ts	All API calls + TypeScript interfaces. MUST stay in sync with backend/schemas.py.
state/portfolio.tsx	React context: accounts (source of truth), holdingsOverride (price-refresh), targets, derived holdings. setAccounts() auto-clears holdingsOverride. refreshHoldings() updates prices without clearing accounts.
utils/schwabParse.ts	Schwab CSV parser + parseTemplateCsv + holdingsForAccount + accountsFromHoldings.
utils/assetClass.ts	9 sub-class constants, PARENT_OF, PARENTS, display colors.

utils/money.ts	fmtMoney(), fmtPct() formatters.
utils/download.ts	Shared downloadText() Blob-download helper (used by TradeTable + SetupPage).
utils/inflation.ts	deflatePoints() for real-vs-nominal chart adjustment.
pages/SetupPage.tsx	CSV upload, account-type override, target allocation, auto-classify unknowns.
pages/DashboardPage.tsx	Main analysis view. All controls debounce 1s to avoid API storms. Refs used for slider/band values to avoid stale closure in debounced callbacks.
pages/SnapshotPage.tsx	Save/load encrypted snapshots + local JSON download.
components/ProjectionChart.tsx	Recharts ComposedChart: area fan + median line + dashed benchmark.
components/TradeTable.tsx	Trade list with Export CSV + Print buttons.
components/BenchmarkControl.tsx	Presets + custom stock/bond benchmark. BENCHMARK_PRESETS lives here.
components/DiscographyPanel.tsx	N/A - belongs to MusicCollection, not this app.

6. Database (Supabase PostgreSQL)

The app shares a Supabase PostgreSQL instance with the MusicCollection app. All tables are prefixed `rebalancer_` to avoid conflicts. No Alembic - migrations are applied via `main.py:run_migrations()` on every startup.

6.1 Tables

Table	Purpose
rebalancer_ticker_tags	Maps ticker -> asset_class + tax_efficiency + expense_ratio. Shared across all users. ~66 seeded ETFs; unknown tickers added on upload.
rebalancer_snapshots	Encrypted portfolio snapshots. Columns: id, pin_hash, payload (Fernet-encrypted JSON), label, created_at, user_id (nullable), description.
rebalancer_users	Email + salted PIN hash for multi-user snapshot ownership.

rebalancer_price_cache	Last-known price per ticker (ticker PK, price, as_of). TTL: 24h.
rebalancer_ai_cache	Gemini output keyed by sha256(kind + portfolio summary JSON). TTL enforced in the router (~24h).

6.2 Migration Pattern

New **tables**: add a SQLAlchemy model class. `create_tables()` in the lifespan startup calls `Base.metadata.create_all` automatically.

New **columns on existing tables**: add an `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` line to `run_migrations()` in `main.py`. Runs on every startup and is idempotent.

Type changes: run a manual `ALTER TABLE ... TYPE ...` against Supabase *before* the deploy lands. The app will 500 if the type doesn't match what the ORM expects.

7. External Services

7.1 GitHub

Repo: github.com/johnlatona-hoh/portfolio-rebalancer, branch: main.

GitHub Actions CI runs on every push: backend pytest (Python 3.13) + frontend tsc/build (Node 20). See [.github/workflows/ci.yml](#).

Both Vercel and Render are connected to main. A git push triggers auto-deploy for Vercel (reliable) and may trigger Render (unreliable - use deploy hook).

7.2 Vercel (Frontend)

Setting	Value
Project URL	https://frontend-sooty-three-ybjekuhhod.vercel.app
Git branch	main
Root Directory	frontend (CRITICAL - must be set or builds fail)
Build command	npm run build
Output directory	dist
Env var	VITE_API_URL = https://portfolio-rebalancer-tc4h.onrender.com

Vercel auto-deploys on every push to main. `deploy.ps1` verifies by watching the live bundle hash change (no Vercel CLI or token needed).

If Root Directory is not set to 'frontend', Vercel looks for package.json at the repo root, finds nothing, and fails. This was the original bug when the project was created via CLI instead of git-connected.

7.3 Render (Backend)

Setting	Value
Service URL	https://portfolio-rebalancer-tc4h.onrender.com
Plan	Free tier (sleeps after ~15 min idle; cold start ~30-60s)
Build command	pip install -r requirements.txt
Start command	uvicorn main:app --host 0.0.0.0 --port \$PORT
Source directory	backend/
Git branch	main
Deploy Hook	Create in Render dashboard > Settings > Deploy Hook. Export as RENDER_DEPLOY_HOOK.

Render's git auto-deploy is unreliable. `deploy.ps1` always POSTs the `RENDER_DEPLOY_HOOK` to force a fresh build, then polls `/openapi.json` for a schema marker.

***Logic-only changes caveat:** the schema marker check cannot detect changes that don't add a new API field. For those, do a behavioral spot-check on the live backend after deploy. Bump `$BACKEND_MARKER_PROP` in `deploy.ps1` whenever a schema field is added.*

7.4 Supabase (Database)

Supabase provides the PostgreSQL instance. Connection string format:

```
postgresql+asyncpg://user:password@host:5432/dbname
```

The Supabase dashboard is at app.supabase.com. Use the SQL editor there for manual type-change migrations. The database is shared with MusicCollection - all rebalancer tables are prefixed `rebalancer_`.

7.5 Google Gemini AI

Detail	Value
SDK	google-genai (NOT the deprecated google-generativeai package)
Model	gemini-2.5-flash
API Key	GEMINI_API_KEY in backend/.env and Render dashboard
Features	Portfolio insights (advisor.py), ticker tag suggestions (tags.py)
Caching	AI cache table caches output keyed by sha256(kind+portfolio-summary). TTL ~24h.
Retry logic	<code>_generate_with_retry()</code> in services/ai.py - exponential backoff on 429.
No-op behavior	All AI features silently no-op when GEMINI_API_KEY is not set.

7.6 Yahoo Finance (Price Data)

Live price refresh uses the Yahoo Finance v8 chart endpoint (no API key required, works from Render's IP). URL pattern:

```
https://query1.finance.yahoo.com/v8/finance/chart/{ticker}
```

Results are cached in `rebalancer_price_cache` for 24 hours. Stooq and Yahoo v7 quote endpoints are blocked or dead from Render's datacenter IP.

8. Deployment

8.1 The Verified Deploy Script (deploy.ps1)

Always use `deploy.ps1` from the repo root. Do not rely on auto-deploy alone.

```
pwsh ./deploy.ps1 -Message 'what changed' # full gate + push + verify
pwsh ./deploy.ps1 -Message '...' -SkipTests # skip backend pytest (faster)
pwsh ./deploy.ps1 -VerifyOnly # just check what's live now
```

The script does:

1. TypeScript type-check (`npx tsc --noEmit`) - aborts on error.
2. Backend parse-check (`ast.parse`) for each `.py` file.
3. Backend pytest (unless `-SkipTests`).
4. `git add -A` + `git commit` + `git push`.
5. Wait for Vercel auto-deploy (polls bundle hash until it changes, 5 min timeout).
6. POST `RENDER_DEPLOY_HOOK` to force a Render rebuild.
7. Poll `/openapi.json` for `$BACKEND_MARKER_PROP` until it appears (15 min timeout).

8.2 Schema Change Checklist

Before deploying a backend schema change

If adding a new column to an existing table: add `ALTER TABLE` line to `run_migrations()` in `main.py`.

If changing a column type: run `ALTER TABLE ... TYPE ...` in Supabase SQL editor BEFORE pushing.

If adding a new response field: bump `$BACKEND_MARKER_PROP` in `deploy.ps1` to the new field name.

If adding a new table: just add the SQLAlchemy model class - `create_tables()` handles the rest.

Keep frontend `api/client.ts` in sync with backend/`schemas.py` for every response shape change.

8.3 CI / GitHub Actions

File: `.github/workflows/ci.yml`. Runs on every push and pull request:

- Backend job: checkout, Python 3.13, pip install, pytest tests/ -v.
- Frontend job: checkout, Node 20, npm install, npx tsc --noEmit, npm run build.

9. Extending the App

9.1 Adding a New Asset Sub-Class

- Add the name to ASSET_CLASSES list in backend/constants.py.
- Add a SUBCLASS entry (parent, tax).
- Add return assumptions to RETURN_ASSUMPTIONS.
- Add a MAX_DRAWDOWN_PCT entry.
- Add CORRELATION entries for each pair with existing classes.
- Add a FEE_ASSUMPTIONS entry.
- Add to the PLACEMENT dicts for all three account types.
- Add to ASSET_CLASSES array in frontend/src/utils/assetClass.ts.
- Add a color to the color map in assetClass.ts.

9.2 Adding a New Backend Endpoint

- Create or edit a router file in backend/routers/.
- Add Pydantic request/response schemas to backend/schemas.py.
- Include the router in main.py with app.include_router(...).
- Mirror the API call and TypeScript interface in frontend/src/api/client.ts.
- Write tests in backend/tests/.

9.3 Adding a New Frontend Dashboard Control

Follow the existing pattern (StrategySlider, DriftBandControl, HorizonControl):

- Add a state variable and a ref (if the control feeds a debounced callback).
- Add the control to resetSettings() in DashboardPage.tsx.
- Pass the value to runAnalysis() or the relevant API call.
- If the control changes the analysis result, add it to the analyzePortfolio() call.
- Debounce any fast-changing inputs (~1s) using analyzeDebounceRef.

9.4 Rebalancing Engine Test Pattern

The engine is pure Python. Add tests in backend/tests/test_rebalance.py:

- Define holdings as dicts: {account_name, account_type, ticker, quantity, cost_basis, current_value}.
- Define tags as {ticker: {asset_class, tax_efficiency}}.
- Call analyze(holdings, targets, tags, gain_aversion=0.0, drift_band_pct=0.0).
- Assert on result['trades'], result['blended'], result['grade'], etc.

10. Return Assumptions (Monte Carlo)

Conservative long-run estimates. Edit RETURN_ASSUMPTIONS in backend/constants.py to tune. These are not predictions.

	Annual Std Dev	Worst Drawdown

11. Known Gotchas & Operational Notes

- **Python 3.13 only** - 3.14 breaks asyncpg and pydantic-core wheel installs.
- **ASCII-only backend source** - Windows imports .py as cp1252; non-ASCII chars in string literals become mojibake in API responses.
- **loadPortfolio() clears accounts** - this is intentional for snapshot restore, but do NOT use loadPortfolio() for price refresh (use refreshHoldings() instead, which preserves the accounts list).
- **Render cold start** - free tier sleeps after 15 min idle. First request takes 30-60s. The WarmupBanner component handles UX.
- **Logic-only deploys** - the deploy.ps1 schema marker check cannot detect changes that don't add a new API field. Always do a behavioral spot-check after a logic-only backend deploy.
- **Render deploy hook not in bash env** - the RENDER_DEPLOY_HOOK env var is a Windows user variable; it is available in PowerShell but not in Git Bash. Run deploy.ps1 via pwsh.
- **BENCHMARK_PRESETS in frontend** - benchmark presets live in frontend/src/components/BenchmarkControl.tsx, not in backend/constants.py.
- **cost_basis == 0 means unknown** - not a 100% loss. Tax-loss harvesting only flags holdings where cost_basis > 0 and current_value < cost_basis.
- **Bands freeze per-account, not portfolio-wide** - within_band_classes() pins frozen classes to their current per-account holdings. Do not use a portfolio-level renormalization approach (that was the original bug, now fixed with multi-account tests).